
Menu-Driven Matrix Operations

Sum of Elements, Row Sums and Transpose of a 2D Array

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that performs the following menu-driven operations on a given two-dimensional array (matrix):

1. Find the sum of all elements in a matrix.
2. Find the sum of each row of a matrix and store it in an array.
3. Find the transpose of a matrix.
4. Exit.

2. Concepts Used

- **Two-dimensional arrays** to store the matrix in row-major form (`a[row][column]`).
- **Nested for loops** to traverse every row and column of the matrix.
- **switch-case** inside a **do-while** loop to build a repeating menu.
- **Functions** to keep each operation separate and reusable.
- **One-dimensional array** to store the sum of each row.
- **Input validation** for the matrix size and for the menu choice.

3. Algorithm

Main Program

1. Start.
2. Read the number of rows (r) and columns (c) of the matrix.
3. Read the $r \times c$ elements of the matrix into array `a[ ][ ]`.
4. Display the menu and read the choice.
5. If choice = 1, perform Operation 1. If choice = 2, perform Operation 2. If choice = 3, perform Operation 3. If choice = 4, exit. Otherwise print an error message.
6. Repeat Steps 4 and 5 until the choice is 4.
7. Stop.

Operation 1: Sum of All Elements

1. Set `sum = 0`.
2. For $i = 0$ to $r - 1$ and for $j = 0$ to $c - 1$, do `sum = sum + a[i][j]`.
3. Print `sum`.

Operation 2: Sum of Each Row

1. For $i = 0$ to $r - 1$, set `rowSum[i] = 0`.
2. For $j = 0$ to $c - 1$, do `rowSum[i] = rowSum[i] + a[i][j]`.
3. Print the sum of every row, then print the `rowSum[ ]` array.

Operation 3: Transpose

1. For $i = 0$ to $r - 1$ and for $j = 0$ to $c - 1$, do $t[j][i] = a[i][j]$.
2. Print $t[][]$ as a matrix with c rows and r columns.

4. Functions Used

Function	Purpose
readMatrix()	Reads all elements of the matrix from the user.
sumOfAll()	Returns the sum of all elements of the matrix.
rowSums()	Calculates the sum of every row and stores it in the rowSum[] array.
transpose()	Stores the transpose of the matrix in a second array $t[][]$.
main()	Reads the size, shows the menu and calls the required function.

5. Source Code

```
#include <stdio.h>

#define MAX 10

/* Reads the matrix elements from the user */
void readMatrix(int a[MAX][MAX], int r, int c)
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

/* Operation 1: sum of all elements */
int sumOfAll(int a[MAX][MAX], int r, int c)
{
    int i, j, sum = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            sum += a[i][j];
    return sum;
}

/* Operation 2: sum of each row, stored in rowSum[] */
void rowSums(int a[MAX][MAX], int r, int c, int rowSum[MAX])
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        rowSum[i] = 0;
        for (j = 0; j < c; j++)
            rowSum[i] += a[i][j];
    }
}

/* Operation 3: transpose of the matrix */
void transpose(int a[MAX][MAX], int r, int c, int t[MAX][MAX])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            t[j][i] = a[i][j];
}

int main()
```

```

{
    int a[MAX][MAX], t[MAX][MAX], rowSum[MAX];
    int rows, cols, choice, i, j;

    printf("Enter the number of rows and columns: ");
    if (scanf("%d %d", &rows, &cols) != 2 ||
        rows < 1 || rows > MAX || cols < 1 || cols > MAX)
    {
        printf("Invalid size. Rows and columns must be between 1 and %d.\n", MAX);
        return 1;
    }

    printf("Enter the matrix elements:\n");
    readMatrix(a, rows, cols);

    do
    {
        printf("\n----- MENU ----- \n");
        printf("1. Sum of all elements\n");
        printf("2. Sum of each row (stored in an array)\n");
        printf("3. Transpose of the matrix\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input. Exiting.\n");
            break;
        }

        switch (choice)
        {
            case 1:
                printf("Sum of all elements = %d\n", sumOfAll(a, rows, cols));
                break;

            case 2:
                rowSums(a, rows, cols, rowSum);
                printf("Row sums:\n");
                for (i = 0; i < rows; i++)
                    printf("Sum of row %d = %d\n", i + 1, rowSum[i]);
                printf("The row-sum array contains: [");
                for (i = 0; i < rows; i++)
                {
                    printf("%d", rowSum[i]);
                    if (i < rows - 1)
                        printf(", ");
                }
                printf("]\n");
                break;

            case 3:
                transpose(a, rows, cols, t);
                printf("Transpose of the matrix:\n");
                for (i = 0; i < cols; i++)
                {
                    for (j = 0; j < rows; j++)
                        printf("%d\t", t[i][j]);
                    printf("\n");
                }
                break;

            case 4:
                printf("Exiting the program. Goodbye!\n");
                break;

            default:
                printf("Invalid choice! Please enter a number from 1 to 4.\n");
        }
    } while (choice != 4);

    return 0;
}

```

6. Sample Output

The program was compiled with GCC (no warnings) and tested with a 2×3 matrix:

```
Enter the number of rows and columns: 2 3
Enter the matrix elements:
1 2 3
4 5 6

----- MENU -----
1. Sum of all elements
2. Sum of each row (stored in an array)
3. Transpose of the matrix
4. Exit
Enter your choice: 1
Sum of all elements = 21

----- MENU -----
1. Sum of all elements
2. Sum of each row (stored in an array)
3. Transpose of the matrix
4. Exit
Enter your choice: 2
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]

----- MENU -----
1. Sum of all elements
2. Sum of each row (stored in an array)
3. Transpose of the matrix
4. Exit
Enter your choice: 3
Transpose of the matrix:
1      4
2      5
3      6

----- MENU -----
1. Sum of all elements
2. Sum of each row (stored in an array)
3. Transpose of the matrix
4. Exit
Enter your choice: 4
Exiting the program. Goodbye!
```

Verification: $1 + 2 + 3 + 4 + 5 + 6 = 21$. Row 1: $1 + 2 + 3 = 6$. Row 2: $4 + 5 + 6 = 15$. The transpose of a 2×3 matrix is a 3×2 matrix, as printed above.

7. Conclusion

The program successfully performs all the required matrix operations through a menu-driven interface. It uses two-dimensional arrays with nested loops and separates each task into its own function, which makes the code easy to read and reuse. The size is limited to 10×10 , and invalid sizes or menu choices are handled with proper messages. Through this assignment I learned how to handle two-dimensional arrays, pass them to functions, and build a repeating menu using do-while and switch-case.